

Nonparametric Regression Using Clusters

Hrishikesh D. Vinod¹ · Fred Violle¹

Accepted: 13 June 2017 / Published online: 19 June 2017
© Springer Science+Business Media, LLC 2017

Abstract We present a fundamentally unique method of nonparametric regression using clusters and test it against classically established methods. We compare two nonlinear regression estimation packages called ‘NNS’, Violle (NNS: nonlinear nonparametric statistics, 2016), and ‘np’, Hayfield and Racine (J Stat Softw 27(5):1–32, 2008), with the help of a simulation using deterministic (DT) and stochastic (ST) regressor models. We find the respective coefficients of determination (R^2) are close for DT models, while finding an advantage to NNS in ST and large sample cases. Regression coefficients are sometimes regarded as approximations to partial derivatives, especially in social sciences. Then, NNS alone has the ability to compute a range of partials evaluated at points within the sample and also out-of-sample. Thus NNS can provide a viable alternative to kernel based nonparametric regressions without using bandwidths for smoothing.

Keywords Curve fitting · Derivative estimation · Partitioning without smoothing · Sufficiency · Perfect fit · Simulation

1 Introduction

Smoothing of histograms to yield Nadaraya–Watson kernel densities was developed in the 1960s. Since regressions are expectations of conditional densities, nonparametric regressions have been applied in Economics and surveyed in Vinod and Ullah (1993)

✉ Hrishikesh D. Vinod
vinod@fordham.edu

Fred Violle
fred.viole@gmail.com

¹ 441 E Fordham Rd., Bronx, NY 10458, USA

and Li and Racine (2007). The ‘np’ package of R, Hayfield and Racine (2008), implements kernel regressions including estimation of partial derivative functions defined at each data point. Vinod (2008), Section 8.4 uses an average of partial derivative estimates as regression coefficients (approximations for marginal propensities or elasticities) desired in some econometric applications. Unfortunately, kernel regression estimates are sensitive to a bandwidth parameter and their performance leaves much to be desired, as we shall show with examples and simulations.

This paper evaluates a newer and fundamentally distinct alternative to kernel regressions. Nonlinear Nonparametric Statistics (NNS) package (maintained by one of us, Viole (2016b)), is explained in Viole and Nawrocki (2012b). In behavioral finance, the concept of loss aversion is modeled by studying lower partial moments of partitioned densities since Bawa (1975) and Vinod and Reagle (2005). Viole and Nawrocki (2012a) prove that aggregating all partial moment matrices equals the covariance matrix, providing much more disaggregated and nuanced information than possible with traditional summary statistics.

Inspired by financial applications, NNS performs hierarchical and partitional clustering using partial moment quadrant means and joins them to yield a nonparametric fitted regression function, $f(x)$. In curve fitting, an extreme overfitting option gives $f(x)$ by simply joining all points. Similarly, in a k-means clustering limit, Bock (2008), each data point makes its own cluster.

In its default setting, NNS starts with this perfect fit and reduces the number of clusters based on the signal to noise ratio in $[0,1]$ provided by a new NNS dependence measure. Total dependence between variables will call for a perfect NNS fit. NNS offers a parameter whereby a specific number of iterative partial moment quadrant partitions can be called. The following discussion walks through an iterative example of NNS progressing in orders of partitions.

1.1 NNS Partition Progression

Figure 1 having two vertical panels provides a visualization of how NNS fits linear segments to a sine wave.¹ The left hand panel has three figures partitioned with respect to (wrt) both axes. The right panel also has three figures, depicting the linear connection of an adaptive sequence of means illustrated in the panel to the immediate left. Note that the algorithm is simply connecting the two endpoints by straight lines, not fitting any linear regression over that subset of observations. The adaptive sequence involves computing the mean of each subset (only if the subset is not empty), further partitioning the data, then computing the means of subsequent partitions.

The first partition occurs at data means, $\bar{X}$, $\bar{Y}$. The point having these coordinates connected to the endpoints comprise the line segments of our ‘Order 1’ segmented partitioning approximation to the underlying nonlinear function (sine wave) depicted in the top right figure. We display higher order approximations along the second and third row of the figure.

¹ In R, we used the variables: $x = \text{seq}(0, 4 * \pi, \pi / 1000)$; $y = \sin(x)$

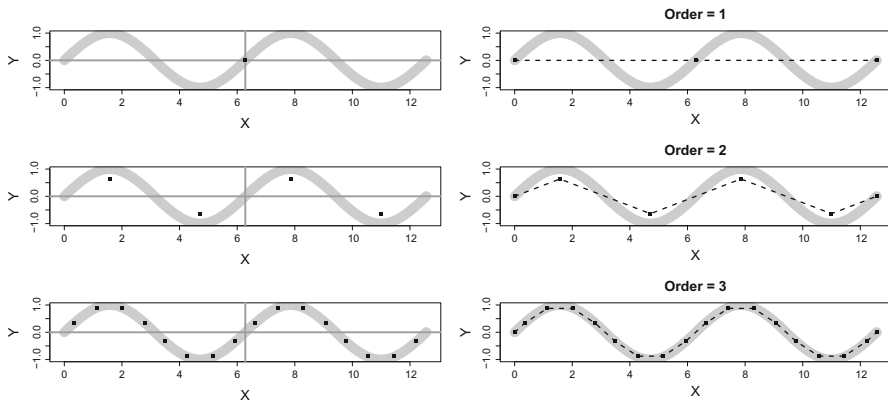


Fig. 1 Visualization of joint distribution $f(x, \sin(x))$ using $4^{(O-1)}$ quadrants, where $order = O = 1, 2, 3$, depicting quadrant means (left panels) and corresponding NNS regression line segments (right panels)

Since the mean of $y = \sin(x)$ is zero for the given range of x the top figure on the left has a straight grey line representing the initial y -axis partition. The first partition results in four quadrants wrt the means of the two variables. These quadrants are seen along the top row of Fig. 1, where the initial partition is called ‘Order = 1’ partition.

Our second order partition further partitions the joint distribution, $f(X, Y)$. Each quadrant from the top row is then further divided into 4 quadrants. The subset means of each of the following 4 quadrant subsets are the 2nd order regression points depicted as black points in the left side middle panel. The ‘Order 2’ curve fitting approximation is seen on the right side middle panel. The bottom right panel of Fig. 1 depicts ‘Order 3’ partition approximation, which is fairly close to the sine curve.

The underlying NNS clustering algorithm computes local averages, not any explicit local least squares fitting using polynomials or splines. Of course, local average can be interpreted as a regression coefficient when a column vector of data are regressed on a column vector of ones.

NNS offers two types of partitioning: the first is performed by both X and Y quadrant means. The joint mean coordinate of that subset is the regression point. This is the default setting. The second type of partition, activated by a (type = “XONLY”) parameter, performs the partitioning along the X -axis only. $\bar{Y}$ of that entire vertical “bandwidth” is the fitted regression point, loosely similar in theory to kernel based nonparametric regressions using a uniform kernel over a given bandwidth.

The endpoints are treated differently in NNS than in other nonparametric regressions. NNS uses a nonlinear dependence measure, [Viole \(2016a\)](#), (`NNS.dep` function in NNS package) between X and Y to determine the endpoints. If the dependence is ‘large’ (in some sense, as defined by the algorithm), then the algorithm fixes the endpoints to the boundary observations. At the other extreme, if no dependence exists, the medians of Y values in each of the boundary partitions are used as robust alternatives to using possibly dubious endpoints.²

² There is considerable literature devoted to the topic of boundaries and endpoints in ‘np’, with no consensus as to how to best handle them.

1.2 Theoretical Reasons Behind NNS Curve Fitting

Let us provide some further discussion of the theory behind NNS' ability to fit any curve. The beauty of the NNS method is that it yields an increasingly improved curve fitting approximation using linear segments (which serve as local linear fits) by increasing the 'Order' of segmentation. Stone (1961) argues that local least squares estimates approximation is best in the *overall* least squares sense. Bellman (1961) provides dynamic programming solutions to Stone's examples. The linear segments (defined over $4^{(O-1)}$ quadrants, where O denotes the order) together create a sequence of line segments comprising an approximation to the nonlinear curve.

An important benefit of such line segments is that they provide readily estimated segment slopes, providing simple and direct estimates of partial derivatives ($\partial y / \partial x$), without involving any infinitesimal changes. Another important NNS advantage appears to be that the axis segmentation is conditional, whereas the bandwidth used in 'np' is fixed. This conditional partitioning offers parsimony by avoiding unnecessary quadrants when no local observations exist. For example, the bottom left panel of Fig. 1 has points only along the sine curve. The quadrants having no points are omitted without hurting the fit. Hence, NNS partitioning is un-phased even when operating on severely clustered data.

Weierstrass' (1885) famous *Approximation Theorem* states that any continuous function $f(x)$ can be approximated by arbitrarily closely by a polynomial $p_n(x)$, of a sufficiently high degree (n). That is, given a compact set K so that $x \in K$, and $\epsilon > 0$, there exists an n offering a "close" approximation defined by:

$$d(f, p_n) = \sup_{x \in K} |f(x) - p_n(x)| < \epsilon. \quad (1)$$

High degree polynomial approximations are rarely (if ever) used in nonlinear regressions because of estimation problems due to collinearity among regressors of various orders and because fitted high order polynomials are ill-behaved, having unpredictable behavior between observed points. NNS retains a similarly desirable limiting behavior with a perfect fit for a sufficiently large value of O , but uses well-behaved linear segments between observed points.

Remark 1: NNS offers a perfect, not approximate, limiting fit to $f(x)$ denoted by $f_O(x)$, for a finitely large order parameter O . That is, given a compact set K so that $x \in K$, there exists a finitely large O satisfying

$$d(f, f_O) = \sup_{x \in K} |f(x) - f_O(x)| \rightarrow 0. \quad (2)$$

The proof is straightforward. We start with observed values of x and $f(x)$. As O increases, the number of quadrants and line segments joining quadrant means increase exponentially according to $4^{(O-1)}$, until each observation becomes its own quadrant mean in the limit. Then, $f(x) = f_O(x)$ must hold, implying a perfect fit in the limit.

Remark 2: The usual caveats of overfitting still apply, and we do not suggest specifying a high order without sufficient reasons. Accordingly, a large O is sometimes (not always) beneficial for partial derivative estimation and/or for increased accuracy

of out-of-sample predictions. In all sets of experiments, the default setting of NNS order is high only when the ‘dependence’ is 1. We later note the relationship between dependence and partial derivative average percentage errors.

A summary of important reasons behind the success of NNS is listed next.

- (i) *Sufficiency*: NNS uses partitions of data based on a carefully designed scheme of partitions. The computed means and variances of each partition rely on the terminology of lower and upper partial moments. The algorithm assigns a partition to each data point while insisting that summing the weighted partitions always preserves the overall mean and variance.³ Thus NNS relies on “sufficient” statistics such as the mean and variance of the data in its partitions.⁴
- (ii) *Gap-free Segments*: Stone’s method is based on approximating curves via “a series broken line segments”. NNS essentially overlaps the segments by sequentially connecting the mean points of the partitions, leaving no gaps, as illustrated in Fig. 1. If we try to use a least squares fit of the form $y = a + bx$, say, over the same data within a partition, there will be separate nonzero intercepts for each partition, often leading to broken line segments.
- (iii) *Perfect Limiting Fit*: NNS has a limit condition described in Remark 1 whereby every observation will be its own fitted regression point implying a perfect fit *in the limit* as we increase the order O . k -means clustering shares this limit condition whereby at the extreme case we will have all observations as their own cluster.⁵

2 Computation of Partial Derivatives

NNS has an embedded partial derivative function, “dy.dx” which retrieves slopes from the linear segments generated from the regression routine and evaluates the derivative for a given point as well as an overall derivative. The overall derivative is the NNS approximation to a linear regression coefficient, and weights each linear segment by the number of observations in that quadrant. The ‘np’ package, by contrast, offers gradients at each point and the overall derivative needs to be computed by averaging those gradients.

Figure 2 illustrates the 6th order NNS regression used to estimate partial derivative of $\sin(x)$ evaluated at $x = 6$. We know the analytic derivative of $\sin(x)$ is $\cos(x)$, and the higher the *order* parameter of the NNS regression, the more accurate the numerical partial derivative. NNS by default provides the full set of local coefficients. The coefficient value represents the slope (derivative) of the line segment in the relevant range of X values.

³ The mean of course is defined as the sum of weighted observations, weighted by 1 over the number of all observations. The single observation weighting and quadrant partitions eventually lead to the limit condition of NNS partitioning.

⁴ The variance is not directly used in NNS regression. [Viole and Nawrocki \(2012a\)](#) demonstrate the covariance matrix equivalence from NNS partitioning and it is mentioned here in order to further exemplify NNS partitioning’s robustness.

⁵ The use of explicit linear regressions within segments is avoided by NNS since it cannot reach such limits. This is because one needs a minimum number of observations for a sensible regression fit. Empty segments are ignored by NNS.

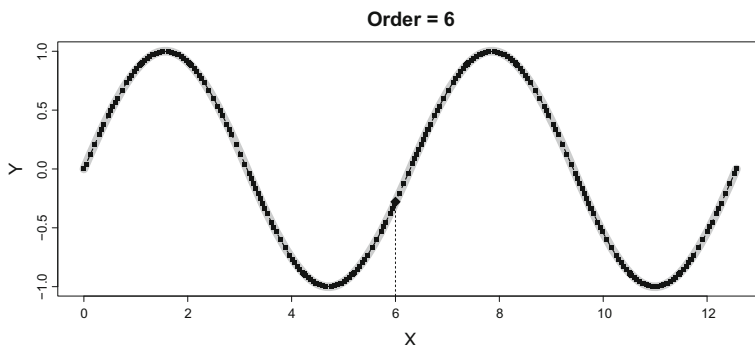


Fig. 2 6th order partition for $y = \sin(x)$ regression with $x = 6$ highlighted with large diamond point. While a 6th order regression has a maximum of 4^5 partitions, only 262 segments are used due to empty quadrants containing no observations.

In the following truncated output, our evaluation point $x = 6$ has a coefficient of 0.9599, which is seen to be very close to the analytically known derivative value, $\cos(6) = 0.9602$. NNS also has an option to return the fitted value for Y directly with the R-command:

```
NNS.reg(x,y,point.est=6)$Point.est
```

(Fitted Value = -0.2794 vs. $\sin(6) = -0.2794$).

	Coefficient	X.Lower.Range	X.Upper.Range
1:	0.95716764	5.9878756	5.9910172
2:	0.95807251	5.9910172	5.9941588
3:	0.95896792	5.9941588	5.9973004
4:	0.95985387	5.9973004	6.0004420
5:	0.96073034	6.0004420	6.0035836
6:	0.96159734	6.0035836	6.0067252
7:	0.96245484	6.0067252	6.0098667

3 Simulation Description

We performed three sets of 6 experiments with varying regressor types and non-linearities comparing (a) the goodness-of-fit or R^2 values, (b) estimated regression coefficients as partial derivatives of conditional expectation function wrt the (noiseless) regressor, and (c) estimated regression coefficients as partial derivatives with increasing orders of noise in the regressor.

For simulations (a) and (b) the chosen sample sizes were: $n = 30, 50, 100, 200, 300$. These were repeated $reps = 1000$ times. Each iteration and sample was unique due to the seed being deterministically set for every $reps$ repetition. The exceptional computational cost associated with 'np' is noteworthy. A quick comparison of the default regressions for our X and Y variables below reveals that NNS elapsed time along the

first row is orders of magnitude faster than the ‘np’ elapsed time along the second row below:

```
> system.time(NNS.reg(x,y)); system.time(npreg(y~x))
```

User	System	Elapsed
0.72	0.03	0.75
User	System	Elapsed
30.87	0.02	31.55

Next, let us compare the goodness-of-fit. The R^2 subroutine found in both packages is from [Doksum and Samarov \(1995\)](#):

$$R^2 = \frac{[\sum_{i=1}^n (y_i - \bar{y})(\hat{y}_i - \bar{y})]^2}{\sum_{i=1}^n (y_i - \bar{y})^2 \sum_{i=1}^n (\hat{y}_i - \bar{y})^2}. \quad (3)$$

Note that this measure of goodness-of-fit is identical to the standard measure when the model is linear and does not suffer from the possibility of negative values that plagues computing R^2 from decomposing the sum of squares [Hayfield and Racine \(2008\)](#).

The sample size for the partial derivative noise test was set to $n = 300$ for all orders of noise. The noise term denoted by ϵ is a standard normal deviate with a random uniform mean with another uniform deviate added, $N(U_1, \frac{1}{m}) + U_2$, whose scale is altered by dividing by m for $m = 160, 80, 40, 20, 10, 1$. Again, all experiments are repeated $reps = 1000$ times.

This experiment touches on another important subject, the signal to noise ratio. NNS uses an internal dependence measure to ascertain the (signal:noise) and restricts the order O accordingly. In high dependence situations, NNS is designed to use a higher *order* and fit the regression with increased number of shorter linear segments. In low dependence situations, NNS will utilize a lower *order* to avoid overfitting. As important as the signal to noise identification is, we leave it for further discussion to not detract from the presentation of NNS regression features.

The following sections contain the test cases used for comparing NNS and ‘np’ in terms of R^2 values and for evaluating NNS partial derivatives. The default settings for both methods are used. The first two experiments of each set contain the deterministic regressors, while the remaining four have random uniform or normally distributed regressors based on sample size n .

3.1 R^2 Experiments with Varying Nonlinearities

We identify some nonlinearities as deterministic (DT) or stochastic (ST) for discussion purposes.

1. Time regressor, DT: $X = \{1, 2, 3, \dots, n\}$ and $Y = 3 + 4x + \epsilon$
2. Time Quadratic, DT: $X = \{1, 2, 3, \dots, n\}$ and $Y = 3 + 4x - 3x^2 + \epsilon$

3. Uniform exponential-1, ST: X has n uniform random numbers and $Y = 3^{1+2x+3x^2} + \epsilon$
4. Uniform exponential-2, ST: X has n uniform random numbers and $Y = e^{0.1+0.2x} + \epsilon$
5. Uniform exponential-3, ST: X has n uniform random numbers and $Y = \frac{1}{e^{1+2x}} + \epsilon$
6. Normal exponential, ST: X has n normal random numbers and $Y = \frac{1}{(1+e^{0.1-0.2x})} + \epsilon$

3.1.1 R^2 Simulation Results

The following Tables 1, 2, 3, 4 and 5 represent the output from the R^2 comparisons with their corresponding parameters. Column 1 represents the NNS R^2 which was achieved by its automatic selection of *order*. The second column presents the standard 'np' regression function R^2 . The reader's attention is directed to the sign of third column reporting the difference of NNS R^2 from 'np'. A positive sign suggests superiority of NNS over 'np'. The fourth column reports the dependence NNS generated and used in its *order* determination. The means of all iterations of R^2 are used, including the differences.

We can summarize the results of the simulation by focusing on the column entitled (NNS R^2 –np R^2) measuring the difference between the two R^2 values. In all five tables together there are 30 comparisons. We find that there are 15 positives, 12 negatives

Table 1 Simulation of R^2 where $n = 30$

	NNS R^2	np R^2	(NNS R^2 –np R^2)	Dependence
DT1	0.999633	0.999859	–0.000226	0.995722
DT2	0.999987	1.000000	–0.000013	0.925926
ST1	0.945300	0.995530	–0.050229	0.787228
ST2	0.239469	0.110863	0.128606	0.735021
ST3	0.238447	0.115796	0.122651	0.735473
ST4	0.233114	0.116380	0.116734	0.729694

Table 2 Simulation of R^2 where $n = 50$

	NNS R^2	np R^2	(NNS R^2 –np R^2)	Dependence
DT1	0.999898	0.999904	–0.000006	0.991473
DT2	0.999999	1.000000	–0.000001	0.965731
ST1	0.981404	0.997617	–0.016213	0.767143
ST2	0.111510	0.074603	0.036907	0.525216
ST3	0.116186	0.077267	0.038919	0.527390
ST4	0.100496	0.074609	0.025887	0.525243

Table 3 Simulation of R^2 where $n = 100$

	NNS R^2	np R^2	(NNS R^2 -np R^2)	Dependence
DT1	0.999981	0.999958	0.000022	0.999999
DT2	1.000000	1.000000	-0.000000	0.999586
ST1	0.999422	0.999282	0.000140	0.922643
ST2	0.069884	0.040527	0.029357	0.481300
ST3	0.072127	0.042474	0.029653	0.483128
ST4	0.062734	0.037664	0.025070	0.490957

Table 4 Simulation of R^2 where $n = 200$

	NNS R^2	np R^2	(NNS R^2 -np R^2)	Dependence
DT1	0.999995	0.999986	0.000010	0.999999
DT2	1.000000	1.000000	-0.000000	0.999441
ST1	0.999833	0.999765	0.000069	0.921304
ST2	0.023161	0.024535	-0.001373	0.327911
ST3	0.025049	0.026559	-0.001511	0.328852
ST4	0.022367	0.023123	-0.000756	0.336586

Table 5 Simulation of R^2 where $n = 200$

	NNS R^2	np R^2	(NNS R^2 -np R^2)	Dependence
DT1	1.000000	0.999993	0.000007	1.000000
DT2	1.000000	1.000000	-0.000000	0.999476
ST1	0.999917	0.999876	0.000041	0.921179
ST2	0.015423	0.017731	-0.002308	0.265499
ST3	0.017145	0.020206	-0.003062	0.265666
ST4	0.012683	0.016686	-0.004003	0.272665

and 3 zeros suggesting an advantage of NNS over 'np'. The purpose of comparing R^2 values is to assure the potential user of NNS that she is not sacrificing goodness-of-fit. Both methods clearly offer appropriate fits and the differences are minor. Further, given NNS's ability to perfectly fit any $f(x)$, the true objective of this simulation was to demonstrate the ability of NNS to avoid overfitting. Given the relatively similar results, we conclude NNS, like 'np', is properly compensating for the noise present.

Slightly better fits for 'np' under deterministic (DT) regressors along the first two rows are noted, but the numbers in the third column are small. In any case, real world applications rarely have deterministic regressors.

4 Partial Derivative Experiments

Now we turn to simulations involving partial derivatives. These are best described with the help of an example of computing the derivative for (a) the function $y = 3 + 4x$

without noise, and (b) for $y = 3 + 4x + \epsilon$, where ϵ denotes a noise term. In both cases we know analytically that $(\partial y / \partial x = 4)$. The simulation considers the performance of NNS in both cases and reports the results in Sects. 4.4 and 4.4.1. Both use `dy.dx` function in NNS and report mean absolute percent error (MAPE) from the known true value of the derivative.

The following code illustrates a short and convenient user interface for NNS derivative computation. We also present the ‘np’ syntax for gradient extraction. NNS derivative without noise:

```
> x=1:300; y=3+4*x; dy.dx(x,y)
[1] 4
```

‘np’ derivative without noise:

```
> gradients(npreg(tydat=y,txdat=x,exdat=median(x),
  gradients=TRUE))
[,1]
[1,] 32.99325
```

NNS derivative with noise:

```
> set.seed(123); x=1:300; y=3+4*x+rnorm(300); dy.dx(x,y)
[1] 4.07978
```

‘np’ derivative with noise:

```
> gradients(npreg(tydat=y,txdat=x,exdat=median(x),
  gradients=TRUE))
[,1]
[1,] 4.08493
```

For consistency, all derivatives for all tests were evaluated at the median of x for the given sample.

4.1 Noiseless Experiments

1. Time regressor, DT: $X = 1, 2, 3, \dots, n$ and $Y = 3 + 4x$
2. Time Quadratic, DT: $X = \{1, 2, 3, \dots, n\}$ and $Y = 3 + 4x - 3x^2$
3. Uniform exponential-1, ST: X has n uniform random numbers and $Y = 3^{1+2x+3x^2}$
4. Uniform exponential-2, ST: X has n uniform random numbers and $Y = e^{.1+.2x}$
5. Uniform exponential-3, ST: X has n uniform random numbers and $Y = \frac{1}{e^{1+2x}}$
6. Normal exponential, ST: X has n normal random numbers and $Y = \frac{1}{(1+e^{1-.2x})}$

4.2 Experiments with Additive Noise

The additive noise terms in the following experiments are normally distributed, $\epsilon = N(U_1, \frac{1}{m}) + U_2$ with a random uniform mean (U_1) added to another random uniform number (U_2) sampled from $[-1, 1]$. The scale factor $m \in [1, 160]$, is such that a smaller m means greater additive noise in the error term of the model.

-
- Springer

panel of Fig. 1 for a visualization of a 1st order NNS regression where $O = 1$ is seen to imply an almost linear regression.

4.4 Noiseless Partial Derivative Simulation Results

The following tables (Tables 6, 7, 8, 9, 10) report the performance of the partial derivatives evaluated at the median for the noiseless experiments in Sect. 4.1. Column 1 is the mean of $\partial X/\partial Y$ for all iterations, while column 2 is the mean of NNS partial derivatives using the regression coefficients, again, for all iterations. In column 3 entitled “NNS MAPE” we report the mean absolute percentage error. Column 4 is the mean of NNS partial derivatives using the finite step (FS) method from `NNS.reg` point estimates surrounding the evaluation point, and column 5 is the NNS finite step (FS) method MAPE. Column 6 is the NNS generated dependence measure. The last three columns report comparable results for ‘np’, showing that ‘np’ MAPE can be huge.

Focusing attention on the columns 3, 5 and 7 (MAPE) in Tables 6, 7, 8, 9 and 10, it is clear that NNS estimates of partial derivatives in columns 3 and 5 are very close to the analytically known true values compared to ‘np’ in columns 7, especially for DT regressors. The NNS-MAPE is very low indeed (never exceeding 1%) in Tables 8, 9, and 10, while ‘np’-MAPE does exceed 1% many times. We note in passing that for rows marked DT and ST in Tables 6, 7, 8, 9 and 10 the finite step NNS(FS)-MAPE

Table 6 Experiments from Sect. 4.1 where $n = 30$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	74.0000	4.0000	0.0000	4.0000	0.0000	344.4349	85.1087	1.0000
DT2	-89.0000	-91.0000	0.0225	-89.2258	0.0025	-524.1021	4.8888	0.9259
ST1	138.8910	129.4437	0.0680	144.6399	0.0414	1723.6976	11.4104	0.9368
ST2	0.2444	0.2712	0.1094	0.2445	0.0001	0.3627	0.4839	0.9956
ST3	-0.2730	-0.2031	0.2561	-0.2732	0.0008	-0.5034	0.8438	0.9727
ST4	-0.2212	-0.2155	0.0259	-0.2213	0.0003	-0.2527	0.1422	0.9842

Table 7 Experiments from Sect. 4.1 where $n = 50$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.0000	4.0000	0.0000	4.0000	0.0000	52.9529	12.2382	1.0000
DT2	-149.0000	-151.0000	0.0134	-149.0392	0.0003	-5209.0988	33.9604	0.9558
ST1	127.5384	115.6244	0.0934	129.6123	0.0163	605.1253	3.7447	0.9274
ST2	0.2444	0.2727	0.1157	0.2444	0.0001	0.3115	0.2744	0.9962
ST3	-0.2721	-0.1914	0.2967	-0.2725	0.0014	-0.4802	0.7650	0.9706
ST4	-0.2209	-0.2077	0.0598	-0.2210	0.0003	-0.2285	0.0340	0.9833

Table 8 Experiments from Sect. 4.1 where $n = 100$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.0000	4.0000	0.0000	4.0000	0.0000	37.6441	8.4110	1.0000
DT2	-299.0000	-299.0000	0.0000	-298.9901	0.0000	-1742.3475	4.8272	0.9996
ST1	121.3185	116.2576	0.0417	122.2464	0.0076	293.1723	1.4166	0.9997
ST2	0.2444	0.2730	0.1170	0.2444	0.0000	0.2492	0.0197	1.0000
ST3	-0.2708	-0.2270	0.1618	-0.2709	0.0004	-0.3002	0.1087	1.0000
ST4	-0.2208	-0.2051	0.0715	-0.2208	0.0000	-0.1956	0.1145	1.0000

Table 9 Experiments from Sect. 4.1 where $n = 200$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.0000	4.0000	0.0000	4.0000	0.0000	91.2045	21.8011	1.0000
DT2	-599.0000	-599.0000	0.0000	-598.9055	0.0002	-27701.6523	45.2465	0.9994
ST1	117.6017	112.9206	0.0398	118.3176	0.0061	189.7930	0.6139	0.9996
ST2	0.2444	0.2717	0.1116	0.2444	0.0000	0.2479	0.0144	1.0000
ST3	-0.2703	-0.2278	0.1571	-0.2704	0.0004	-0.2935	0.0859	1.0000
ST4	-0.2210	-0.2031	0.0809	-0.2210	0.0000	-0.2006	0.0924	1.0000

Table 10 Experiments from Sect. 4.1 where $n = 300$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.0000	4.0000	0.0000	4.0000	0.0000	32.9932	7.2483	1.0000
DT2	-899.0000	-899.0000	0.0000	-899.0000	0.0000	-5021.4227	4.5856	0.9995
ST1	115.9423	113.9901	0.0168	116.5996	0.0057	178.6084	0.5405	0.9996
ST2	0.2443	0.2599	0.0637	0.2443	0.0000	0.2389	0.0222	1.0000
ST3	-0.2705	-0.2467	0.0877	-0.2706	0.0004	-0.2737	0.0120	1.0000
ST4	-0.2209	-0.2102	0.0489	-0.2209	0.0000	-0.2033	0.0798	1.0000

in column 5 seems to be always the smallest, implying superior partial derivative estimates.

4.4.1 Experiments with Added Noise in Error Terms

Tables 11, 12, 13, 14, 15 and 16 contain the results from experiments having the additive noise term. How much noise is too much noise? For the DT regressors (experiments DT1 and DT2), noise did not materially affect the output. For experiments reported along lines marked ST1 to ST4 in Tables 11, 12, 13, 14, 15 and 16, the combination of noise in the error term in conjunction with the presence of stochastic regressors

Table 11 Experiments from Sect. 4.2 where $m = 160$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.000	3.957	0.0108	3.997	0.0008	17.445	3.361	1.000
DT2	-899.000	-899.001	0.0000	-899.002	0.0000	-22281.398	23.785	0.999
ST1	115.942	130.652	0.1269	116.996	0.0091	163.911	0.414	0.978
ST2	0.244	0.238	0.0277	0.237	0.0282	0.175	0.285	0.261
ST3	-0.270	-0.277	0.0234	-0.278	0.0273	-0.246	0.091	0.265
ST4	-0.221	-0.213	0.0374	-0.213	0.0374	-0.174	0.213	0.306

Table 12 Experiments from Sect. 4.2 where $m = 80$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.000	3.957	0.0108	3.997	0.0009	17.211	3.303	1.000
DT2	-899.000	-899.001	0.0000	-899.002	0.0000	-21642.443	23.074	0.999
ST1	115.942	118.592	0.0229	117.000	0.0091	164.150	0.416	0.978
ST2	0.244	0.237	0.0290	0.237	0.0295	0.176	0.278	0.261
ST3	-0.270	-0.283	0.0464	-0.284	0.0503	-0.246	0.090	0.265
ST4	-0.221	-0.211	0.0463	-0.211	0.0463	-0.174	0.210	0.307

Table 13 Experiments from Sect. 4.2 where $m = 40$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.000	3.957	0.0109	3.997	0.0009	17.223	3.306	1.000
DT2	-899.000	-899.001	0.0000	-899.002	0.0000	-20654.175	21.975	0.999
ST1	115.942	117.355	0.0122	116.997	0.0091	164.174	0.416	0.978
ST2	0.244	0.226	0.0764	0.226	0.0748	0.177	0.276	0.261
ST3	-0.270	-0.272	0.0070	-0.273	0.0106	-0.242	0.107	0.265
ST4	-0.221	-0.212	0.0406	-0.212	0.0406	-0.174	0.214	0.307

generated too much variability around the median point for NNS to generate reliable partial derivative estimates.

The performance of NNS partial derivative estimation can be judged by focusing on MAPE columns (3, 5 and 7) in Tables 11, 12, 13, 14, 15 and 16. The error percentages NNS-MAPE are all well below 1% except once for ST1. The highly stochastic experiments ST2:ST4 actually exhibit improved errors as the noise level increases. This somewhat counterintuitive behavior can be explained by the deteriorating dependence forcing a lower order regression. Again, NNS finite step method outperformed NNS coefficients in all experiments.

Table 14 Experiments from Sect. 4.2 where $m = 20$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.000	3.957	0.0108	3.997	0.0008	11.688	1.922	1.000
DT2	-899.000	-899.001	0.0000	-899.002	0.0000	-22890.365	24.462	0.999
ST1	115.942	116.776	0.0072	116.956	0.0087	163.870	0.413	0.977
ST2	0.244	0.225	0.0790	0.226	0.0732	0.180	0.263	0.260
ST3	-0.270	-0.285	0.0535	-0.285	0.0548	-0.236	0.126	0.267
ST4	-0.221	-0.205	0.0723	-0.205	0.0723	-0.175	0.209	0.305

Table 15 Experiments from Sect. 4.2 where $m = 10$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.000	3.958	0.0104	3.996	0.0009	13.703	2.426	1.000
DT2	-899.000	-899.001	0.0000	-899.002	0.0000	-23031.020	24.618	0.999
ST1	115.942	110.657	0.0456	116.896	0.0082	163.779	0.413	0.977
ST2	0.244	0.222	0.0909	0.223	0.0875	0.192	0.215	0.262
ST3	-0.270	-0.281	0.0399	-0.281	0.0394	-0.232	0.142	0.266
ST4	-0.221	-0.214	0.0326	-0.214	0.0326	-0.176	0.205	0.301

Table 16 Experiments from Sect. 4.2 where $m = 1$

	dy/dx	NNS dy/dx	NNS MAPE	NNS (FS) dy/dx	NNS (FS) MAPE	np dy/dx	np MAPE	NNS dep
DT1	4.000	3.920	0.020	3.994	0.0014	3.995	0.001	1.000
DT2	-899.000	-898.997	0.000	-899.004	0.0000	-22704.342	24.255	0.999
ST1	115.942	-91.929	1.793	115.477	0.0040	132.660	0.144	0.921
ST2	0.244	0.229	0.063	0.230	0.0598	0.134	0.452	0.265
ST3	-0.270	-0.289	0.067	-0.289	0.0672	-0.220	0.188	0.266
ST4	-0.221	-0.228	0.033	-0.228	0.0332	-0.166	0.249	0.274

The last column in many tables here reports the NNS computed ‘dependence’ measure for signal to noise ratio. Certain sophisticated scatter plots can also reveal the nature of signal:noise in a given data. The default setting of NNS chooses the *order* parameter O based on the dependence measure. Of course, NNS also lets the user choose O . Our simulations reveal that a lower O measures the partial derivative along a longer line segment, diminishing the sensitivity of partial derivative estimates, if desired in noisy applications.

While further research is needed to improve this performance even further, we are not aware of any other method for estimating partial derivatives of complicated

nonlinear relations at specified points having completely unknown and unspecified nonparametric functional forms from non-experimental data.

5 Out-of-Sample Predictions

An important distinguishing feature of NNS over 'np' is the ability to obtain out-of-sample predictions well beyond the observed range, if needed. Using our sequence of X in the earlier sine wave example which ended at 4π , the next X value would be $4\pi + \pi/1000$. The following output shows that the out-of-sample predicted value is remarkably well estimated for our nonlinear example, without knowing the functional form.

```
> NNS.reg(x,y,point.est = 4*pi+(pi/1000),order='max')
$Point.est
[1] 0.003141587
```

the 'np' predicted value is:

```
> eval=data.frame(x=4*pi+(pi/1000))
> predict(npreg(y~x), newdata =eval)
[1] -4.898425e-16
```

where the actual value is:

```
sin(4*pi+(pi/1000))=0.003141587
```

The estimate can be extended further, visually represented below in Fig. 4 for $x = 13$.

```
> NNS.reg(x,y,point.est = 13,order='max')$Point.est
[1] 0.4336287
```

the 'np' predicted value is:

```
> eval=data.frame(x=13)
> predict(npreg(y~x), newdata = eval)
[1] 0
```

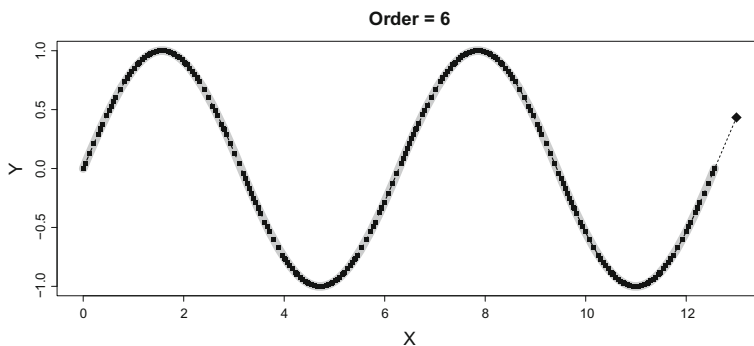


Fig. 4 Extension of range and estimate in NNS regression

where the actual value is:

```
> sin(13)
[1] 0.420167
```

Obviously, extending a linear endpoint estimate over extended ranges will ultimately be ineffectual for nonlinear processes. However, the use of the nonlinear regression for sequential points allows NNS to integrate these insights into an ARMA model (NNS . ARMA), capable of incorporating nonlinear periodic behavior.

6 Summary and Discussion of Results

We explain the NNS algorithm which starts with the choice of a small integer O representing the *order*. Next, it partitions the scatterplot data into $4^{(O-1)}$ quadrants, while fitting unbroken sequence of line segments by ignoring quadrants with zero observations in an adaptive fashion. The unknown nonlinear function is fitted by these line segments while preserving the two ‘sufficient’ statistics, namely, the overall mean and variance. Sufficiency, gap-free segments and perfect limiting fit are three properties of NNS which together explain why it can offer superior performance in most situations.

Since the NNS approach to curve-fitting is sensitive to the presence of noise, the default NNS algorithm chooses the O parameter by using a signal to noise ratio quantified by a ‘dependence’ measure.

Our simulation results show that for sample sizes greater than 50, or for stochastic regressor models, NNS provides a better fit than ‘np.’ One possible explanation for the outperformance of NNS’ ability to fit the stochastic regressors is absence of kernel smoothing. While smoothing is a critical component of ‘np,’ it is absent in NNS.

One can argue at a philosophical level that regression coefficients are *not* partial derivatives. But, NNS’ technique essentially bypasses the controversy by going back to basic calculus. It offers an alternative estimate of the partial derivative by using the well-known ‘finite step method’ of numerical differentiation. Accordingly, one expects that the point prior and point after the point of interest, x , represent the $f(x - h)$ and $f(x + h)$ respectively, where the magnitude of the ‘presumed infinitesimal’ change, h , is constant. This is indeed the case when NNS coefficients are used on deterministic (DT) regressor models for any suitable choice of the *order*, including high orders.

NNS regression coefficients as well as partial derivative estimates are both somewhat less accurate when O is relatively large in *noisy* stochastic (ST) regressor cases. We believe that such behavior of NNS algorithm is entirely appropriate, given the internal *order* reduction NNS performs. It reminds the researcher that in addition to valid specification, reasonably high signal to noise ratio is also needed when one interprets regression coefficients as partial derivatives.

Our simulations report that the presence of noise does not vitiate the NNS coefficient partial derivative accuracy in the DT case. We also find that in the absence of noise, the NNS estimates of partials of ST models are fairly accurate. Thus, it is the joint presence of stochastic regressor model with noise that is responsible for NNS’ apparent inability to generate exact estimates of partial derivatives, even though the percent error (MAPE) remains mostly below 1% for experiments of very high dependence

(‘dependence’ > 0.95). The MAPE in lower dependence situations can be reduced significantly by choosing a lower O parameter or by using the finite step alternative. While a higher value of O may be desirable for a superior fit, a smaller O may be better for derivative estimation in noisy ST models.

Overall, NNS is able to fit functions with deterministic and stochastic regressors without knowing the nonlinear functional forms. NNS is also able to accurately generate partial derivatives under a majority of scenarios, and offers a distinct out-of-sample range extension. Thus, our results demonstrate that NNS is a robust and appealing alternative to nonparametric kernel based regressions, worthy of further analysis and research.⁶

Acknowledgements We would like to thank the anonymous referees providing valuable commentary on earlier versions of this manuscript.

References

- Bawa, V. S. (1975). Optimal rules for ordering uncertain prospects. *Journal of Financial Economics*, 2(95), 121.
- Bellman, R. (1961). On the approximation of curves by line segments using dynamic programming. *Communications of the ACM*, 4(6), 284. <http://dl.acm.org/citation.cfm?doid=366573.366611>.
- Bock, H.-H. (2008). Origins and extensions of the k -means algorithm in cluster analysis. *Journal électronique d'Histoire des Probabilités et de la Statistique* [electronic only], 4(2), 14–181418. <http://eudml.org/doc/130880>.
- Doksum, K., & Samarov, A. (1995). Nonparametric estimation of global functionals and a measure of the explanatory power of covariates in regression. *Annals of Statistics*, 23(5), 1443–1473. <https://projecteuclid.org/euclid.aos/1176324307>.
- Hayfield, T., & Racine, J. S. (2008). Nonparametric econometrics: The np package. *Journal of Statistical Software*, 27(5), 1–32. <http://www.jstatsoft.org/v27/i05/>.
- Li, Q., & Racine, J. S. (2007). *Nonparametric econometrics*. Princeton: Princeton University Press.
- Stone, H. (1961). Approximation of curves by line segments. *Mathematics of Computation*, 15(73), 40–47. <http://www.jstor.org/stable/2283327>.
- Vinod, H. D. (2008). *Hands-on intermediate econometrics using R: Templates for extending dozens of practical examples*. Hackensack, NJ: World Scientific. ISBN 10-981-281-885-5. <http://www.worldscibooks.com/economics/6895.html>.
- Vinod, H. D., & Reagle, D. (2005). *Preparing for the worst: Incorporating downside risk in stock market investments (monograph)*. New York: Wiley.
- Vinod, H. D., & Ullah, A. (1993). General nonparametric regression estimation and testing in econometrics. In G. S. Maddala, C. R. Rao, & H. D. Vinod (Eds.), *Handbook of statistics: Econometrics* (Vol. 11, pp. 85–116). New York: North Holland, Elsevier.
- Viole, F. (2016a). *Beyond correlation: Using the elements of variance for conditional means and probabilities*. SSRN eLibrary. <http://ssrn.com/abstract=2745308>.
- Viole, F. (2016b). *NNS: Nonlinear nonparametric statistics*. R package version 0.3.3. <https://CRAN.R-project.org/package=NNS>.
- Viole, F., & Nawrocki, D. (2012a). *Cumulative distribution functions and UPM/LPM analysis*. SSRN eLibrary. <http://ssrn.com/abstract=2148482>.
- Viole, F., & Nawrocki, D. (2012b). *Deriving nonlinear correlation coefficients from partial moments*. SSRN eLibrary. <http://ssrn.com/abstract=2148522>.

⁶ The R code for simulation is available on GitHub: <https://github.com/OVVO-Financial/NNS/tree/Data-and-Simulation-Routines/NNS-Simulation-Routines> The NNS R routines are also available on GitHub: <https://github.com/OVVO-Financial/NNS/tree/NNS-Beta-Version/R>